National University of Singapore

CS1101S — Programming Methodology

AY2023/2024 Semester 1

Midterm Assessment

Time allowed: 1 hour 30 minutes

QUESTION PAPER

INSTRUCTIONS

1. This **QUESTION PAPER** contains **24** Questions in **7** Sections, and comprises **14** printed pages, including this page.
2. The **ANSWER SHEET** comprises **4** printed pages.
3. Use a pen or pencil to **write** your **Student Number** in the designated space on the front page of the **ANSWER SHEET**, and **shade** the corresponding circle **completely** in the grid for each digit or letter. DO NOT WRITE YOUR NAME!
4. You must **submit only** the **ANSWER SHEET** and no other documents. Do not tear off any pages from the ANSWER SHEET.
5. All questions must be answered in the space provided in the **ANSWER SHEET**; no extra sheets will be accepted as answers.
6. Write legibly with a **pen** or **pencil** (do not use red color). Untidiness will be penalized.
7. For **multiple choice questions (MCQ)**, **shade** in the **circle** of the correct answer **completely**.
8. The full score of this assessment is **100** marks.
9. This is a **Closed-Book** assessment, but you are allowed to bring with you one double-sided **A4 / letter-sized sheet** of handwritten or printed **notes**.
10. Where programs are required, write them in the **Source §2** language. A **reference** of some of the **pre-declared functions** is given in the **Appendix** of the Question Paper.
11. In any question, unless it is specifically allowed, your answer **must not use functions** given in, or written by you for, other questions.

Section A: List and Box Notations [8 marks]

(1) [4 marks] (MCQ)

What is the result of evaluating the following Source program in ***box notation***?

```
const lst = list(1,2);
pair(head(lst), pair(tail(lst), tail(tail(lst))));
```

A. `[1, [2, null]]`
B. `[[[1, 2], null], null]`
C. `[[1, 2, null], null]`
D. `[[1, [2, [null]]], null]`
E. `[1, [[2, null], null]]`
F. It is impossible to express the result in box notation.

(2) [4 marks] (MCQ)

What is the result of evaluating the following Source program in ***list notation***?

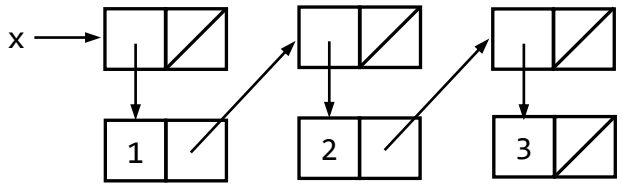
```
const lst = list(1,2);

function through(curr) {
    return is_null(curr)
           ? lst
           : pair(tail(curr), through(tail(curr)));
}

through(lst);
```

A. `list(list(2), null, list(1, 2))`
B. `list(list(2), null, 1, 2)`
C. `list([2, null], list(1, 2))`
D. `list(1, null, 1, 2)`
E. It is impossible to express the result in list notation.
F. The program leads to an error.

Section B: Box and Pointer Diagrams [12 marks]

(3) [4 marks] (MCQ)

Which of the following programs produces the pairs shown in the following box-and-pointer diagram?



A.
```
const x = list(list(1, list(2, list(3))));
```
B.
```
const x = pair(pair(1, pair(2, pair(3, null))), null);
```
C.
```
const x = list(list(1, list(2, 3)));
```
D.
```
const x = list(list(1, 2, list(3)));
```
E.
```
const x = list(1, list(2, list(3)));
```
F.
```
const x = list(list(1, list(2, 3), null));
```

(4) [4 marks]

Draw the box-and-pointer diagram for the value of `x` after the evaluation of the following program. Clearly show where `x` is pointing to.

```
const x = pair(list(pair(1, 1)), list(1));
```

(5) [4 marks]

Draw the box-and-pointer diagram for the value of `x` after the evaluation of the following program. Clearly show where `x` is pointing to.

```
const lst = list(list(pair(null, null)));
const x = accumulate((x, y) => list(y, x), 0, lst);
```

Section C: Data Structures — Dictionaries [9 marks]

A ***dictionary*** data structure provides a way to store data ***entries*** where each entry consists of a ***key*** and a ***value***. For example, student numbers and student names can form the key-value entries of a dictionary. We will represent a key-value entry as a pair in Source, where the head of the pair is the key and the tail is the value. A dictionary is represented as a list of pairs in Source.

The following shows two dictionaries for student names and student exam marks.

```
const student_names = list( pair("A101X", "Alice"),
                            pair("A105Y", "Bob"),
                            pair("A102Z", "Cat") );

const student_marks = list( pair("A101X", 78),
                            pair("A105Y", 69),
                            pair("A102Z", 84) );
```

(6) [5 marks]

Complete the declaration of function `is_entry`, which takes in two arguments: a key and a dictionary, and returns `true` if the dictionary has an entry corresponding to the input key, or returns `false` otherwise. For example,

```
is_entry("A105Y", student_names); // returns true
is_entry("A109W", student_names); // returns false
```

Write your answer only in the dashed-line box(es).

(7) [4 marks]

Complete the declaration of function `remove_entry`, which takes in two arguments: a key and a dictionary, and returns a dictionary with the entry corresponding to the input key removed, if such entry exists in the dictionary. You may assume that the keys are unique in the dictionary. For example,

```
remove_entry("A105Y", student_names);
// returns [["A101X", "Alice"], [["A102Z", "Cat"], null]]
```

Write your answer only in the dashed-line box(es).

Section D: Orders of Growth [16 marks]

For each question in this section, choose the order of growth that best characterize the running time of the function. Note that N is a positive integer value.

(8) [4 marks] (MCQ)

```
function fun(N) {
   return N <= 1
          ? list(1)
          : N % 2 === 0
          ? enum_list(1, N)
          : map(x => enum_list(1, N), enum_list(1, N));
}
```

A. $\Omega(N^2)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N^2)$
E. $\Theta(N^3)$
F. $O(N)$
G. $O(N \log N)$
H. $O(N^2)$

(9) [3 marks] (MCQ)

```
function fun(N) {
   return N <= 1  ?  1  :  N * fun(N / 4);
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(10) [3 marks] (MCQ)

```
function fun(N) {
   return N <= 1  ?  1  :  fun(N - 10) + fun(N - 10);
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(11) [3 marks] (MCQ)

```
function fun(N) {
   if (N <= 1) {
      return 1;
   } else {
      const L = enum_list(1, N);
      return 3 * fun(N - 10);
   }
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

(12) [3 marks] (MCQ)

```
function fun(N) {
   if (N <= 1) {
      return 1;
   } else {
      const L = enum_list(1, math_floor(N));
      return N * fun(N / 2);
   }
}
```

A. $\Theta(1)$
B. $\Theta(\log N)$
C. $\Theta(N)$
D. $\Theta(N \log N)$
E. $\Theta(N^2)$
F. $\Theta(N^2 \log N)$
G. $\Theta(N^3)$
H. $\Theta(k^N)$ where k is some constant greater than 1

Section E: List Shifting [13 marks]

(13) [4 marks]

Complete the declaration of function `circular_left_shift`, which takes as argument a list, and returns a list that is the result of circularly shifting the input list to the left by one position. For example,

```
circular_left_shift(list(3, 4, 5, 6, 7));  // returns list(4, 5, 6, 7, 3)
circular_left_shift(list(3));  // returns list(3)
circular_left_shift(null);  // returns null
```

You must not use the `append` function, and you must not declare any new function. The running time of your function must have an order of growth of $O(n)$, where n is the length of the input list.

Write your answer only in the dashed-line box(es).

(14) [5 marks]

Complete the declaration of function `circular_right_shift`, which takes as argument a list, and returns a list that is the result of circularly shifting the input list to the right by one position. For example,

```
circular_right_shift(list(3, 4, 5, 6, 7));  // returns list(7, 3, 4, 5, 6)
circular_right_shift(list(3));  // returns list(3)
circular_right_shift(null);  // returns null
```

You must not use the `append` function, and you must not declare any new function. The running time of your function must have an order of growth of $O(n)$, where n is the length of the input list.

Write your answer only in the dashed-line box(es).

(15) [4 marks] (MCQ)

Which of the following is a correct way to reverse a list `xs`?

A.	`accumulate((x, ys) => circular_right_shift(pair(x, ys)), null, xs);`
B.	`accumulate((x, ys) => circular_left_shift(pair(x, ys)), null, xs);`

C.	`accumulate(`
	`  (x, ys) => pair(head(ys), circular_right_shift(pair(x, tail(ys)))),`
	`  null, xs);`

D.	`accumulate(`
	`  (x, ys) => pair(head(ys), circular_left_shift(pair(x, tail(ys)))),`
	`  null, xs);`

E.	`accumulate((x, ys) => pair(x, circular_right_shift(ys)), null, xs);`
F.	`accumulate((x, ys) => pair(x, circular_left_shift(ys)), null, xs);`

Section F: Trees [24 marks]

(16) [3 marks] (MCQ)

Consider the following constant declarations for data structures.

```
const data1 = list(1, pair(2, null), list(4, 3));
const data2 = list(pair(1, 2), 3, list(4));
const data3 = list(1, list(2, list(3, list(4))));
```

Which of the above data structures is/are tree(s)?

A. `data1` only
B. `data1`, and `data2` only
C. `data1`, and `data3` only
D. `data2`, and `data3` only
E. `data1`, `data2`, and `data3`
F. None of the data structures

(17) [5 marks] (MCQ)

Consider the function `new_tree_sum` which sums all elements of a tree. Note that it differs from the one shown in lectures.

```
function new_tree_sum(tree) {
    return is_null(tree)
           ? display(0)
           : new_tree_sum(tail(tree))
             +
             ( is_list(head(tree))
               ? new_tree_sum(head(tree))
               : display(head(tree)) );
}
const tree = list(1, list(2, list(3, 4), 5));
new_tree_sum(tree);
```

What is the sequence of values printed by the `display` function when the above program is evaluated?

A. `0 0 0 5 4 3 2 1`
B. `0 0 5 0 4 3 2 1`
C. `0 5 0 0 4 3 2 1`
D. `1 2 3 4 5 0 0 0`
E. `1 2 3 4 0 5 0 0`
F. `1 2 3 4 0 0 5 0`
G. `5 4 3 2 1 0 0 0`
H. `5 0 4 3 0 2 0 1`

(18) [5 marks]

Complete the declaration of function `max_tree`, which takes as argument a tree of positive numbers, and returns the maximum value in that tree. For example,

```
const tree1 = null;
const tree2 = list(1, list(2, list(3, 4), 5));
const tree3 = list(5, list(6, list(3, 2), 1));
max_tree(tree1); // returns 0
max_tree(tree2); // returns 5
max_tree(tree3); // returns 6
```

You must not use any higher-order list processing functions. (You may find the predeclared function `math_max` useful.)

Write your answer only in the dashed-line box(es).

(19) [5 marks] (MCQ)

Recall our implementation of `accumulate_tree` in class:

```
function accumulate_tree(f, op, initial, tree) {
    return accumulate(
             (x, ys) => !is_list(x)
                        ? op(f(x), ys)
                        : op(accumulate_tree(f, op, initial, x), ys),
             initial,
             tree );
}
const tree4 = list(2, list(1, 3, list(5, 4), list(6)));
accumulate_tree(display, (x, y) => x + y, 0, tree4);
```

What is the sequence of values printed by the `display` function when the above program is evaluated?

A. 1 2 3 4 5 6
B. 2 1 3 5 4 6
C. 2 1 3 6 5 4
D. 6 5 4 2 1 3
E. 6 5 4 1 3 2
F. 6 4 5 3 1 2

(20) [6 marks]

Complete the declaration of the function `replace_elem`, which takes in three arguments: an element to be replaced `old_e`, a new element `new_e`, and a `tree`, and returns a tree with all elements `old_e` replaced with `new_e` using the `accumulate_tree` function. For example,

```
const tree5 = list(1, list(4, list(3, 2), 1));
replace_elem(1, 5, tree5); // returns list(5, list(4, list(3, 2), 5))
```

Write your answer only in the dashed-line box(es).

Section G: Functions [18 marks]

(21) [4 marks] (MCQ)

Consider the following function `mystery`:

```
function mystery(x, y) {
    return z => z >= x && z <= y;
}
```

Which one of the following expressions returns `true`?

A. `mystery(5, 6)(10);`
B. `mystery(10, 5)(6);`
C. `mystery(6, 5)(10);`
D. `mystery(5, 10)(6);`
E. `mystery(6, 10)(5);`
F. `mystery(10, 6)(5);`

(22) [5 marks] (MCQ)

Consider the following function `enigma`:

```
function enigma(a) {
    return b => c => c(a(b));
}
```

Which one of the following expressions returns `17`?

A. `enigma(x => x * x)(4)(y => y + 1);`
B. `enigma(x => x + 1)(4)(y => y * y);`
C. `enigma(4)(x => x + 1)(y => y * y);`
D. `enigma(4)(x => x * x)(y => y + 1);`
E. `enigma(x => x + 1)(y => y * y)(4);`
F. `enigma(x => x * x)(y => y + 1)(4);`

(23) [5 marks] (MCQ)

The function `fun_filter` takes as arguments a list of predicates `ps` and a value `x`, and returns the list of predicates in `ps` for which the predicates hold on `x`. For example,

```
const my_fs = list(y => y === 0, y => y < 10,
                   y => y > 20, y => y % 2 === 0);
fun_filter(my_fs, 6); // returns list(y => y < 10, y => y % 2 === 0)
```

Which one of the following is a correct implementation of `fun_filter`?

A. `const fun_filter = (ps, x) => filter(x, ps);`
B. `const fun_filter = (ps, x) => filter(ps, x);`
C. `const fun_filter = (ps, x) => filter(f => x => f(x) ? x : f, ps);`
D. `const fun_filter = (ps, x) => filter(f => f(x) ? f : x, ps);`
E. `const fun_filter = (ps, x) => filter(f => f(x), ps);`
F. `const fun_filter = (ps, x) => filter(f => x => f(x), ps);`

(24) [4 marks] (MCQ)

Mathematicians have been striving for a minimal framework to express computation. In that effort, the mathematician Alonzo Church devised a way to represent numbers using functions: He represented the number 0 by

```
const zero = f => x => x;
```

and the number 3 by

```
const three = f => x => f(f(f(x)));
```

Observe that the Church numeral `three` applies a given function `f` three times to a given value `x`.

The successor function that computes the next Church numeral for a given Church numeral can be expressed by

```
const succ = n => f => x => f(n(f)(x));

succ(three); // returns the Church numeral for 4:
             // a function that is equivalent to
             // f => x => f(f(f(f(x))))
```

The quest for minimalism did not stop with Church's numerals. The mathematicians Moses Schönfinkel and Haskell Curry proved that any computable function can be represented by combining the following functions `S`, `K`, and `I` using function application:

```
const S = x => y => z => x(z)(y(z));
const K = x => y => x;
const I = x => x;
```

For example, the successor function above can be represented the following SKI combination.

```
const succ = S(S(K(S))(K));
```

Observe that `I` is not used in the definition of `succ`. You might wonder if `I` is needed at all in SKI combinations. The answer is no: We can find an SKI combination `J` such that `J` does not use `I` and such that `J(A)` returns `A` for any argument `A`, just like `I` does. Which of the following declarations can replace `I`? For example, `J(three)` should return `three`.

A. `const J = S(S)(S);`
B. `const J = K(S)(K);`
C. `const J = S(K)(K);`
D. `const J = K(S)(S);`
E. `const J = S(S)(K);`
F. `const J = K(K)(K);`
G. `const J = K(K)(S);`

——— END OF QUESTIONS ———

Appendix

The following **LISTS** functions are supported in Source §2:

- `pair(x, y)`: Makes a pair from `x` and `y`.
- `is_pair(x)`: Returns `true` if `x` is a pair and `false` otherwise.
- `head(x)`: Returns the head (first component) of the pair `x`.
- `tail(x)`: Returns the tail (second component) of the pair `x`.
- `is_null(xs)`: Returns `true` if `xs` is the empty list, and `false` otherwise.
- `is_list(x)`: Returns `true` if `x` is a list as defined in the lectures, and `false` otherwise. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of the chain of `tail` operations that can be applied to `x`.
- `list(x1, x2,..., xn)`: Returns a list with `n` elements. The first element is `x1`, the second `x2`, etc. Iterative process; time: $O(n)$, space: $O(n)$, since the constructed list data structure consists of n pairs, each of which takes up a constant amount of space.
- `length(xs)`: Returns the length of the list `xs`. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `map(f, xs)`: Returns a list that results from list `xs` by element-wise application of `f`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `build_list(f, n)`: Makes a list with `n` elements by applying the unary function `f` to the numbers 0 to `n - 1`. Iterative process; time: $O(n)$, space: $O(n)$.
- `for_each(f, xs)`: Applies `f` to every element of the list `xs`, and then returns `true`. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `reverse(xs)`: Returns list `xs` in reverse order. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`. The process is iterative, but consumes space $O(n)$ because of the result list.
- `append(xs, ys)`: Returns a list that results from appending the list `ys` to the list `xs`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `member(x, xs)`: Returns first postfix sublist whose head is identical to `x` (`===`); returns `null` if the element does not occur in the list. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `remove(x, xs)`: Returns a list that results from `xs` by removing the first item from `xs` that is identical (`===`) to `x`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `remove_all(x, xs)`: Returns a list that results from `xs` by removing all items from `xs` that are identical (`===`) to `x`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `filter(pred, xs)`: Returns a list that contains only those elements for which the one argument function `pred` returns `true`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`.
- `enum_list(start, end)`: Returns a list that enumerates numbers starting from `start` using a step size of 1, until the number exceeds (`>`) `end`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`. For example, `enum_list(2, 5)` returns the list `list(2, 3, 4, 5)`.
- `list_ref(xs, n)`: Returns the element of list `xs` at position `n`, where the first element has index 0. Iterative process; time: $O(n)$, space: $O(1)$, where n is the length of `xs`.
- `accumulate(op, initial, xs)`: Applies binary function `op` to the elements of `xs` from right-to-left order, first applying `op` to the last element and the value `initial`, resulting in r_1, then to the second-last element and r_1, resulting in r_2, etc, and finally to the first element and r_{n-1}, where n is the length of the list. Thus, `accumulate(op, zero, list(1,2,3))` results in `op(1, op(2, op(3, zero)))`. Iterative process; time: $O(n)$, space: $O(n)$, where n is the length of `xs`, assuming `op` takes constant time.

Some other functions supported in Source §2:

- `is_boolean(x)`: Returns `true` if `x` is a boolean value, and `false` otherwise.
- `is_number(x)`: Returns `true` if `x` is a number, and `false` otherwise.
- `is_string(x)`: Returns `true` if `x` is a string, and `false` otherwise.
- `display(v)`: Displays the value `v` in the REPL.
- `stringify(v)`: Returns a string that represents the value `v`. For example, `stringify(123)` returns the string `"123"`, and `stringify(false)` returns the string `"false"`.
- `math_floor(x)`: Returns the largest integer that is equal to or less than the number `x`.
- `math_sqrt(x)`: Returns the square root of the number `x`.
- `math_pow(base, exp)`: Returns the result of raising `base` to the power of `exp`.
- `math_min(x1, x2,..., xn)`: Returns the smallest of the `n` arguments. Time: $O(n)$, space: $O(1)$.
- `math_max(x1, x2,..., xn)`: Returns the largest of the `n` arguments. Time: $O(n)$, space: $O(1)$.

——— END OF QUESTION PAPER ———